

SplitwiseSim, Explained: From First Principles to Internals

A complete, self-contained guide to the `splitwise-sim` codebase (Microsoft/Azure Research), covering *why* it exists, *what* it models, and *how* every file fits together — from a beginner's mental model to the exact preemption logic used in the paper's experiments.

Source: github.com/Mutinifni/splitwise-sim Paper: Patel et al., "*Splitwise: Efficient Generative LLM Inference Using Phase Splitting*", ISCA 2024.

Table of Contents

1. [Background: The Problem Splitwise Solves](#)
 2. [What SplitwiseSim Actually Is](#)
 3. [Discrete-Event Simulation, from Scratch](#)
 4. [The 30,000-Foot Architecture](#)
 5. [Configuration System \(Hydra\)](#)
 6. [Core Data Model: Requests as DAGs](#)
 7. [The Hardware Model: Servers, Processors, Interconnects](#)
 8. [Instances: Where Computation Actually Happens](#)
 9. [Schedulers: Deciding Where Requests Go](#)
 10. [The Performance Model: How Durations Are Estimated](#)
 11. [Executors: Walking the Request DAG](#)
 12. [Allocators, Arbiters, Routers: The Outer Loop](#)
 13. [Start States: How a Simulation Begins](#)
 14. [End-to-End Walkthrough: The Life of One Request](#)
 15. [Running a Simulation, Concretely](#)
 16. [Outputs and Metrics](#)
 17. [Advanced: Preemption and Batch Selection Logic](#)
 18. [Advanced: Memory and KV-Cache Accounting](#)
 19. [Extending SplitwiseSim](#)
 20. [File-by-File Reference Map](#)
 21. [Glossary](#)
-

1. Background: The Problem Splitwise Solves

To understand the codebase, you first need to understand the systems problem it was built to study.

1.1 How generative LLM inference works

When you send a prompt to an LLM (say, 500 tokens) and ask it to generate a reply (say, 200 tokens), the GPU work happens in **two distinct phases**:

Phase	Also called	What happens	Compute profile
Prompt phase	"prefill"	All input tokens are processed <i>in parallel</i> in one forward pass to produce the first output token	Compute-bound : high GPU utilization, short in wall-clock time relative to phase 2
Token phase	"decode"	Each subsequent output token is generated one at a time, autoregressively, each requiring a full forward pass that reads the growing KV-cache	Memory-bandwidth-bound : low GPU compute utilization, but this phase usually dominates end-to-end latency because it's serial

These two phases have almost opposite hardware appetites: prompt processing wants raw FLOPs, token generation wants memory bandwidth (to keep re-reading the KV-cache and model weights per token). Yet virtually every inference server (vLLM, ORCA, TensorRT-LLM, etc.) runs **both phases on the same GPU**, in the same continuous batch. This is efficient for GPU occupancy but means you can't tune the hardware independently for each phase, and the two phases interfere with each other inside a batch (a new prompt arriving mid-decode can stall ongoing token generation, or vice versa).

1.2 Splitwise's idea

Splitwise (the technique, not the simulator) proposes: *physically separate* the two phases onto different machines.

- **Prompt machines**: optimized for compute (e.g., newer/faster GPUs), handle only prefill.
- **Token machines**: optimized for memory capacity/bandwidth (e.g., older/cheaper GPUs), handle only decode.
- After the prompt machine computes the first token, it ships the **KV-cache** for that request over a fast interconnect to a token machine, which then continues autoregressive decoding.

This lets you build **heterogeneous clusters** (e.g., pairing a DGX-H100 for prompts with a DGX-A100 for tokens) that get more aggregate throughput per dollar and per watt than homogeneous clusters where every machine does both phases. The paper reports up to 1.4× higher throughput at 20% lower cost, or 2.35× more throughput at the same power/cost budget, versus non-split baselines.

1.3 Why you'd want a simulator for this

Building real heterogeneous GPU clusters to test every possible configuration (cluster mix, scheduler, KV-transfer bandwidth, batch limits, request-rate, ...) is expensive and slow.

SplitwiseSim is the discrete-event simulator the Splitwise authors built to explore this design space quickly, using **profiled performance data** from real DGX-A100/DGX-H100 runs instead of running an actual LLM.

2. What SplitwiseSim Actually Is

SplitwiseSim is a **Python discrete-event simulator (DES)** for LLM-serving clusters. In one sentence: *it replays a trace of incoming requests through a simulated cluster of servers, GPUs, schedulers, and interconnects, and estimates how long everything takes using a performance model, without ever running a real neural network.*

Key properties:

- It is **not** a real inference engine — no tensors are computed. Durations are looked up/interpolated from a CSV of pre-profiled measurements (`data/perf_model.csv`) or set to constants.
 - It uses **Hydra** for hierarchical YAML configuration (cluster shape, scheduler choice, trace file, etc. are all config-driven, overridable from the command line).
 - It can simulate **both** baseline (mixed-batch) LLM serving *and* Splitwise-style phase-split serving, letting you compare them apples-to-apples.
 - It supports **parallel sweeps** over many configurations at once via Ray or joblib, which is how the paper produced its large design-space exploration plots.
 - Output is a set of **CSV files** (summary + per-request + per-node + per-instance-iteration) that downstream notebooks turn into plots and Gantt charts.
-

3. Discrete-Event Simulation, from Scratch

If you've never worked with a DES before, here's the mental model, because SplitwiseSim's entire engine (`simulator.py`) is built on it.

A discrete-event simulator doesn't advance time in small fixed ticks (like a physics engine). Instead:

1. It keeps a **priority queue (min-heap)** of future `Event`s, each tagged with the simulated time at which it should fire.
2. It repeatedly **pops the earliest event**, jumps the simulation clock forward to that event's time, and runs the event's callback (`action`).
3. That callback typically does some bookkeeping and **schedules new future events** (e.g., "in 3.2 seconds, this task will finish").
4. This repeats until the queue is empty or an end time is reached.

Because nothing happens *between* events, the simulation can jump straight from "the batch starts at t=10.0" to "the batch iteration finishes at t=10.043" without simulating every microsecond in between — this is what makes DES fast even over a simulated day (`end_time: 86400` seconds is the default).

3.1 `simulator.py` — the engine

python



```
class Event:
    def __init__(self, time, action):
        self.time = time
        self.action = action          # a zero-arg callable (usually a lambda)
    def __lt__(self, other):
        return self.time < other.time  # so heapq can order Events by time

class Simulator:
    def __init__(self, end_time):
        self.time = 0
        self.end_time = end_time
        self.events = []              # heap of Events
        self.deleted_events = []      # "soft-deleted" events (for cancellation)

    def schedule(self, delay, action):
        if delay == 0:
            action()                  # run immediately, no heap needed
            return None
        event = Event(self.time + delay, action)
        heapq.heappush(self.events, event)
        return event

    def run(self):
        while self.events and self.time < self.end_time:
            event = heapq.heappop(self.events)
            if event in self.deleted_events:
                continue              # was cancelled; skip
            self.time = event.time     # jump the clock
            event.action()              # execute callback, which may schedule mor
```

A global module-level variable `sim` holds the current `Simulator` instance (the code openly comments "bad practice, but it works for now" — a pragmatic singleton so that any object anywhere can call the free functions below without holding a reference).

Convenience wrappers used **everywhere** in the codebase:

python



```
def clock():                                # "what time is it right now?"
def schedule_event(delay, action):          # "do `action` `delay` seconds from now"
def cancel_event(event):                    # "never mind, don't run that"
def reschedule_event(event, delay):         # cancel + schedule again (used to "extend")
```

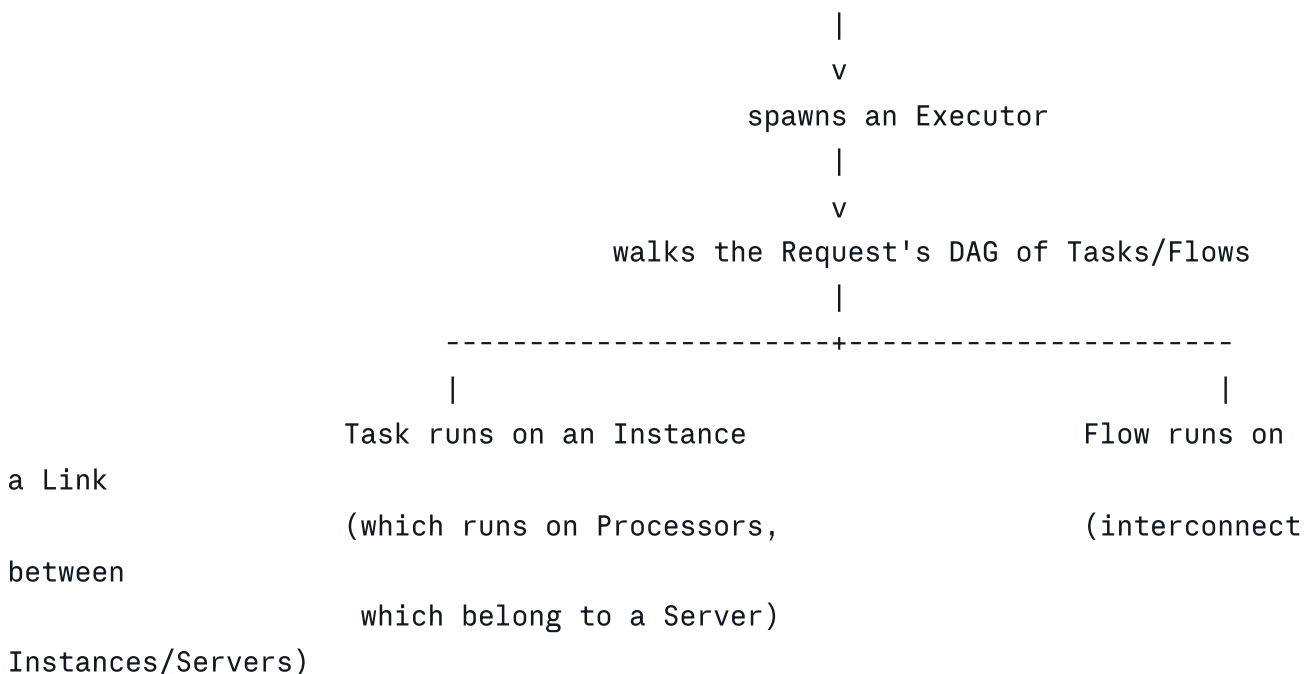
`TraceSimulator(Simulator)` is the concrete subclass actually used: it additionally holds a `Trace` (the list of requests), a `Cluster`, `Application`s, a `Router`, and an `Arbiter`. Its `load_trace()` schedules one arrival `Event` per request in the trace at the request's `arrival_timestamp`. Its `run()` kicks off the cluster/router/arbiter at time 0, then calls the base-class event loop, then calls `save_results()` to dump all the CSV outputs.

Takeaway: almost every class in this codebase (`Task`, `Flow`, `Instance`, `Link`, `Scheduler`...) participates in this event loop by calling `schedule_event(...)` to say "come back to me later," rather than by looping or blocking.

4. The 30,000-Foot Architecture

Requests flow through a small pipeline of objects, each with a clear single responsibility:

Trace (CSV) --arrival--> Router --route--> Application's Scheduler



Mapping this to real-world nouns:

Simulator concept	Real-world analog
Cluster	A datacenter's worth of machines
Server	One physical machine (e.g., a DGX-A100 box)
Processor (CPU)(GPU)	One chip inside a server
Instance	A running copy of a model, occupying one or more GPUs (this is what actually executes requests)
Application	The logical "endpoint" a client hits (has one model, one scheduler, a pool of Instances)
Request	One user's prompt+generation job (has a DAG of Tasks/Flows)
Task (PromptTask)(TokenTask)	The "compute" nodes of a request — literally the prefill or a decode step
Flow (KVCacheTransferFlow)	The "communication" nodes of a request — e.g., shipping the KV-cache
Link (NVLink)(IBLink)/...	A network/interconnect edge that Flows run on
Scheduler	Decides which Instance(s) a Request's Tasks run on
Allocator	Spins Instances up/down on Processors (autoscaling — mostly a no-op today)
Router	Cluster-level request dispatch to the right Application (no-op today, single app)
Arbiter	Cluster-level resource re-balancing between Applications (no-op today)
PerformanceModel	Answers "how long will this batch/task take?" using profiled data
PowerModel	Answers "how much power does this task/processor draw?"

The README states this exact pipeline explicitly (lightly paraphrased): all requests first arrive at a cluster-level Router which forwards them to their target Application; each Application uses Allocators to spin up/down Instances and Schedulers to load-balance Requests across Instances; Requests are represented as a DAG of Task/Flow Nodes, with Tasks running on Instances/Processors and Flows running on Links/interconnects.

5. Configuration System (Hydra)

SplitwiseSim is entirely **config-driven**. Rather than editing Python to change an experiment, you edit or override YAML.

5.1 Top-level `configs/config.yaml`

```
yaml

defaults:
  - cluster: dgx-a100
  - router: noop
  - arbiter: noop
  - hardware_repo: default
  - model_repo: default
  - orchestrator_repo: default
  - performance_model: constant
  - power_model: constant
  - applications: solo
  - trace: test_trace
  - start_state: orca
  - override hydra/launcher: ray
  - _self_

end_time: 86400          # simulate one full day, in seconds
debug: False
seed: 0

output_dir: results/${seed}/${start_state.state_type}/${trace.filename}/...
```

Each line under `defaults` says "pull in `configs/<group>/<name>.yaml` as the default for config group `<group>`." For example `cluster: dgx-a100` means "by default use `configs/cluster/dgx-a100.yaml`," which you can override at the command line with `cluster=half_half` to instead load `configs/cluster/half_half.yaml`.

5.2 The config groups, and what each governs

Config group	Directory	Governs
<code>cluster</code>	<code>configs/cluster/</code>	Which server SKUs, how many of each, interconnect topology, power budget
<code>trace</code>	<code>configs/trace/</code>	Which request-arrival CSV file to replay
<code>router</code>	<code>configs/router/</code>	Cluster-level routing policy (currently only <code>noop</code>)
<code>arbiter</code>	<code>configs/arbiter/</code>	Cluster-level autoscaling policy (currently only <code>noop</code>)
<code>applications</code>	<code>configs/applications/</code>	Which model(s), and which scheduler/allocator each application uses
<code>model_repo</code>	<code>configs/model_repo/</code>	Catalog of model architectures (<code>llama2-70b</code> , <code>bloom-176b</code> , ...) and sizes
<code>orchestrator_repo</code>	<code>configs/orchestrator_repo/</code>	Catalog of available Scheduler and Allocator implementations
<code>hardware_repo</code>	<code>configs/hardware_repo/</code>	Catalog of Processor types, Interconnect types, and full server SKUs
<code>performance_model</code>	<code>configs/performance_model/</code>	<code>constant</code> (fixed durations, for debugging) or <code>db</code> (profiled DB-driven)
<code>power_model</code>	<code>configs/power_model/</code>	How power draw is estimated
<code>start_state</code>	<code>configs/start_state/</code>	How Instances are pre-allocated onto the cluster before t=0

5.3 Instantiation via `_target_`

Many config files carry a special `_target_: some.module.ClassName` field. Hydra's `instantiate(cfg)` uses this to construct the actual Python object, passing the other YAML fields as constructor kwargs. This is why, e.g., changing

`applications.0.scheduler=token_jsq` in the command line swaps in a completely different Python `Scheduler` subclass (`TokenJSQScheduler`), defined via `configs/orchestrator_repo/schedulers/token_jsq.yaml`'s `(_target_)` with zero code changes.

5.4 A concrete example run

From the README, `scripts/run_baseline_h_example.sh`:

```
bash

SCHEDULER=token_jsq
NUM_DGX_A100=0
NUM_DGX_H100=1
START_STATE=baseline
TRACE=test_trace

python run.py \
    applications.0.scheduler=$SCHEDULER \
    cluster=half_half \
    cluster.servers.0.count=$NUM_DGX_A100 \
    cluster.servers.1.count=$NUM_DGX_H100 \
    start_state=$START_STATE \
    performance_model=db \
    trace.filename=$TRACE \
    debug=True \
    seed=0
```

Each override does one thing:

- `cluster=half_half` swaps the cluster config from `dgx-a100.yaml` (all A100s) to `half_half.yaml` (1 A100 box + 1 H100 box by default).
- `cluster.servers.0.count=0` / `cluster.servers.1.count=1` then dials that mix down to *zero* A100 boxes and *one* H100 box — i.e., a single-machine "Baseline-H100" configuration.
- `applications.0.scheduler=token_jsq` swaps the default `round_robin` scheduler for `token_jsq` (join-shortest-queue by pending tokens).
- `start_state=baseline` picks a simple "one instance per server" starting layout instead of the Splitwise-specific split layout.
- `performance_model=db` switches from constant fake durations to the profiled database model.
- `debug=True` turns on the extra per-iteration instance logging used for Gantt charts.

5.5 Parallel sweeps

Hydra's `--multirun` flag, combined with the Ray launcher plugin configured in `config.yaml` (`override hydra/launcher: ray`), lets you sweep many configurations across many cores/machines in one command, e.g. `python --multirun run.py seed=0,1,2,...,9`. The `scripts/` directory ships many pre-built sweep scripts (`run_isocost.sh`, `run_isopower.sh`, `run_throughput.sh`, etc.) that reproduce the paper's experiment grids from `configs/experiment/`.

6. Core Data Model: Requests as DAGs

This is the conceptual heart of the simulator. A `Request` is not just "a prompt string" — it's represented as a **Directed Acyclic Graph (DAG)** of `Node`s, using the `networkx` library for the graph structure. This generality is deliberate: it lets the same engine represent both "simple" requests (prompt → tokens, on one machine) and Splitwise's split requests (prompt → KV-transfer → tokens, on two machines) without special-casing.

6.1 `Node` — the common base (`node.py`)

Both `Task`s and `Flow`s inherit from `Node`. A `Node` is a state machine:

```
NONE → QUEUED → RUNNING → COMPLETED
      |
      v
BLOCKED (preempted) → RUNNING (resumed) → COMPLETED
```

with an `ABORTED` escape hatch from any state. Each transition method (`arrive()`, `run()`, `run_after_preempt()`, `complete()`, `preempt()`, `abort()`) does two things: (1) asserts the current state is valid for this transition, and (2) stamps a timestamp into `self.metrics` (via `clock()`) so that queueing time, service time, blocked time, etc. can be reconstructed after the simulation finishes. This metrics-on-every-transition pattern recurs at the `Request` level too (see `request.py`'s `arrive_at_router`/`arrive_at_scheduler`/`run_on_executor`/`complete_at_scheduler`/`complete_at_router`).

6.2 `Task` — computation nodes (`task.py`)

```
python

class TaskType(IntEnum):
    COMPUTE = 0
    PROMPT = 1
    TOKEN = 2
```

- `PromptTask`: represents prefill. Carries `prompt_size` (number of input tokens). `is_complete()` returns true once exactly 1 token has been generated (the first output

token, produced by prefill). Its `memory` property computes the KV-cache footprint for `prompt_size + 1` tokens.

- `TokenTask`: represents decode. Carries `token_size` (number of tokens *left to generate* after the first one — note `GenerativeLLMRequest.__post_init__` creates it with `token_size=self.token_size - 1`, since token #1 came from the prompt task). `is_complete()` returns true once `generated_tokens == token_size`.

Both track

`processing_tokens`/`processed_tokens`/`generating_tokens`/`generated_tokens` at **iteration granularity** (see §8), because real serving systems don't run a Task to completion in one shot — they run it one continuous-batching iteration at a time, interleaved with other requests' tasks.

6.3 `Flow` — communication nodes (`flow.py`)

python

```
class FlowType(IntEnum):
    DEFAULT = 0
    KVCacheTransfer = 1
```

A `Flow` has a `src` and `dest` (both `Instance`s), a `size` (bytes to move), and runs on a `Link` rather than an `Instance`. `KVCacheTransferFlow` is the type used by Splitwise-style schedulers to move a request's KV-cache from the prompt Instance to the token Instance after prefill completes.

6.4 `GenerativeLLMRequest` (`request.py`)

python

```
@dataclass(kw_only=True)
class GenerativeLLMRequest(Request):
    processed_tokens: int
    generated_tokens: int
    prompt_size: int
    token_size: int
    ...

    def __post_init__(self):
        self.max_seq_len = self.prompt_size + self.token_size
        prompt_task = self.create_task(task_type=TaskType.PROMPT, prompt_size=self.prompt_size)
        token_task = self.create_task(task_type=TaskType.TOKEN, token_size=self.token_size)
        self.dag.add_edge(prompt_task, token_task)
        self.root_node = prompt_task
```

So *by default*, every request's DAG is simply `PromptTask → TokenTask`. Schedulers that want KV-cache transfer (§9) surgically **rewrite this DAG** at scheduling time — removing the direct edge and inserting a `KVCacheTransferFlow` node between them: `PromptTask → KVCacheTransferFlow → TokenTask`. This is exactly the code in `scheduler.py`'s `add_kv_cache_transfer()`.

`Request.from_dict(row)` is the factory used when loading a trace CSV: each row becomes one `GenerativeLLMRequest` (currently the only supported `request_type`, value `2` = `GENERATIVE_LLM`, per the enum in `request.py`).

6.5 A useful method: `estimate_kv_cache_size`

python

```
def estimate_kv_cache_size(self, num_tokens=None, model=None):  
    return 2 * self.batch_size * num_tokens * model.architecture.hidden_size \  
        * model.architecture.num_layers * model.size.dtype_size
```

This is the standard KV-cache formula: $2 \text{ (for Key and Value)} \times \text{batch} \times \text{tokens} \times \text{hidden_size} \times \text{num_layers} \times \text{bytes-per-element}$. It's used both for Instance memory accounting (§18) and for sizing `KVCacheTransferFlow`s.

7. The Hardware Model: Servers, Processors, Interconnects

7.1 `Processor` (`processor.py`)

The lowest-level unit that can *run compute* (a `Task`). Subclassed into `CPU` and `GPU`. Tracks `memory_size`/`memory_used` (raises a logged OOM warning — written to `oom.csv` — if a processor's memory is oversubscribed) and `power`.

7.2 `Server` (`server.py`)

A collection of `Processor`s plus local `interconnects` (e.g., NVLink between the GPUs in a DGX box). Built via `Server.from_config(sku_cfg, server_id=...)`, which reads a SKU's processor list and interconnect list from `hardware_repo` and instantiates each one. Servers track their aggregate `power` via `power_model.get_server_power()`.

7.3 `Link` / `interconnects` (`interconnect.py`)

`Link` is the networking counterpart of `Processor` — it's what `Flow`s (not `Task`s) run on. It has a `bandwidth`, tracks `bandwidth_used`, and computes a Flow's duration as `flow.size / (bandwidth - bandwidth_used)`. Concrete subclasses: `PCIeLink` (`CPU ↔ GPU`), `EthernetLink`/`IBLink` (`Server ↔ Server`), `NVLink`/`RDMADirectLink` (`GPU ↔ GPU`). There's also a special `DummyLink` — infinite flows, bandwidth never actually consumed — used purely to model a fixed transfer *delay* without doing real bandwidth contention bookkeeping (this is what `KVTokenJSQScheduler` uses for KV-cache transfers by default, since exact network topology modeling is marked TODO in the code).

7.4 `Cluster` (`cluster.py`)

Just a dict of SKU-name → list of `Server` objects, plus a list of cross-server `interconnects`, plus a `power_budget`. `Cluster.from_config()` reads `cluster.servers` (a list of `{sku, count}` pairs) and `cluster.interconnects` from YAML and builds everything.

7.5 `hardware_repo.py` / `model_repo.py` / `orchestrator_repo.py`

These three modules are near-identical **repository/registry** patterns: each reads a directory of YAML files at startup (`utils.read_all_yaml_cfgs(path)`) and exposes `get_X(name)` accessors that use Hydra's `instantiate()` to lazily construct objects on demand (e.g., `hardware_repo.get_processor("a100-80gb")`, `model_repo.get_model_architecture("llama2-70b")`, `orchestrator_repo.get_scheduler("token_jsq", ...)`). This indirection is what lets the `cluster`/`applications`/etc. configs refer to hardware and orchestrators **by name** instead of inlining their full definitions everywhere. Each module keeps its singleton in a module-level global (same "global-as-service-locator" pattern as `simulator.sim` and `performance_model.performance_model`).

8. Instances: Where Computation Actually Happens

`instance.py` is the largest and most intricate file in the codebase — this is where realistic LLM-serving scheduling (continuous batching, preemption, memory limits) is actually simulated. There are three levels, each building on the last.

8.1 `Instance` (base class) — one task at a time

The simplest possible model: whenever a `Task` arrives and nothing else is running, run it **to completion** as a single opaque duration from `get_duration()` (not iteration-based). No batching at all. Useful mainly for testing / as a conceptual baseline, and used with `performance_model=constant`.

8.2 `ORCAInstance` — continuous batching (no preemption)

Named after the ORCA paper's iteration-level scheduling ("Orca: A Distributed Serving System for Transformer-Based Generative Models", OSDI '22), which today's real systems (vLLM, TensorRT-LLM, etc.) all implement some variant of. Key ideas:

- Requests are tracked in a `pending_requests` list (one task at a time per request — the *head* of `request_tasks[request]`), plus a `batch` list of Tasks currently executing.
- `select_batch()` greedily adds pending requests' head-tasks into the batch up to `max_batch_size`, respecting per-task memory limits against `max_memory`.
- `start_iteration()`: given the selected batch, calls the performance model's `get_iteration_duration(batch, instance)` to get **one iteration's** wall-clock time, then figures out `get_num_contiguous_iterations()` — how many iterations can run *back-to-back* before anything needs re-evaluating. If the batch is pure decode tasks, it can run `min(token_size - generated_tokens across the batch)` iterations

contiguously without re-selecting the batch (a performance optimization: don't re-simulate every single token-generation step if nothing changes about the batch composition in between). If a prompt task is present, only 1 iteration runs before re-checking (since a completing prefill changes the batch).

- A single `schedule_event(...)` is set for the *entire* contiguous run, calling `complete_iteration()` at the end.
- `pause_iteration()`: if a **new** task arrives mid-flight that should join on the *next* iteration boundary (not immediately, since that would corrupt an in-flight batched matmul), the currently-scheduled completion event is **rescheduled to fire sooner** — at the next natural iteration boundary — via `reschedule_event`. This is the mechanism used to implement "continuous" (a.k.a. iteration-level or in-flight) batching: new requests can join a running batch without waiting for every request in the batch to finish.
- `complete_iteration()`: for each task in the batch, calls `task.complete_iteration()` (bumps its processed/generated token counters) and checks `is_complete()`; completed tasks leave the batch via `task_completion()`; then `start_iteration()` is called again to pick up newly-arrived work and continue.

This does **not** support preemption — once a task is running, it stays in the batch until it completes or the whole batch is re-evaluated at a safe boundary.

8.3 `SplitwiseInstance` — adds preemption + batch-token limits

Adds two new knobs on top of ORCA behavior:

- `max_batch_tokens`: an additional cap besides `max_batch_size` — batching is limited by *total tokens processed this iteration* (prompt tokens count heavily; each decode task contributes 1 token), not just request count. This models a real system's fixed compute/memory budget per iteration.
- `max_preemptions`: a cap on how many times a single task can be preempted before it's guaranteed a slot in the next batch (starvation avoidance).

`task_arrival()` here is smarter: if the instance is already fully busy but a new (typically Prompt) task could still fit within `max_batch_tokens` by displacing decode tasks, it calls `preempt_iteration()`, which just triggers an early `pause_iteration()`-style re-evaluation. Full batch-selection logic is in `select_batch()` (overridden — see §17 for the exact priority ordering it uses).

8.4 `Instance.from_config` — the type switch

```
python
```



```

@classmethod
def from_config(cls, instance_cfg, **kwargs):
    instance_type = instance_cfg.instance_type
    if instance_type == "DEFAULT":
        return Instance(**kwargs)
    elif instance_type == "ORCA":
        return ORCAInstance(max_batch_size=..., **kwargs)
    elif instance_type == "Splitwise":
        return SplitwiseInstance(max_batch_size=..., max_batch_tokens=..., max_

```

So which of the 3 instance behaviors you get is purely a config choice (`instance_type` field in a `start_state` YAML), not a code change.

8.5 Memory bookkeeping on every Instance

Every `Instance` maintains `self.memory` (currently used bytes) and `self.max_memory` (total GPU memory across its `processors`). `alloc_memory(tag, bytes)` / `free_memory(tag, bytes)` are called by `Task.run()` / `Task.complete()` and `Flow.run()` / `Flow.complete()` to reserve/release KV-cache space as tasks execute — see §18.

9. Schedulers: Deciding Where Requests Go

`scheduler.py` implements the **application-level** load-balancing policy: given an incoming `Request`, which `Instance`(s) should its Tasks run on? All schedulers share the base class `Scheduler(ABC)`, and must implement one abstract method:

```

python

@abstractmethod
def schedule(self, request, *args, **kwargs):
    """Assign request's DAG nodes to instance(s)."""

```

9.1 Non-split schedulers (single Instance runs both phases)

- `RandomScheduler`: pick a uniformly random Instance; chain prompt→token so they run back-to-back with no re-queueing overhead (`prompt_task.chain = [token_task]`).
- `RoundRobinScheduler`: cycle through Instances in order (`self.instance_index`).
- `JSQScheduler` ("Join Shortest Queue"): pick the Instance with the fewest `pending_requests`.

All three assign *both* the prompt task and token task to the *same* Instance — this is the "baseline" (non-Splitwise) mode, where mixed batching happens naturally because one

Instance's `ORCAInstance`/`SplitwiseInstance` batch can contain both prompt and token tasks from different requests.

9.2 `KV Scheduler` — the `Splitwise` base class

Introduces the concept of separate `prompt_instances` and `token_instances` pools (an `Instance`'s `tag` — set at start-state time — decides which pool it's added to). Provides the shared helper:

python



```
def add_kv_cache_transfer(self, request, src_instance, dest_instance, bandwidth):
    # rewires the DAG: prompt_task -> KVCacheTransferFlow -> token_task
    # assigns prompt_task to src_instance, token_task to dest_instance
    # attaches a DummyLink of the given bandwidth to the flow
```

This is the literal mechanism implementing phase-splitting: it mutates a request's DAG in-flight, at scheduling time, to insert the cross-machine hop.

9.3 Concrete `Splitwise` schedulers

- `KVTokenJSQScheduler`: picks the least-loaded prompt Instance and least-loaded token Instance (by `sched_pending_tokens`), always ships the KV-cache between them. This is the scheduler behind the `Splitwise-HA` example script in the README (prompt on DGX-H100, tokens on DGX-A100).
- `OverlapKVTokenJSQScheduler`: identical, but multiplies the transfer bandwidth by 10× to *approximate* a real system's ability to overlap 90% of the KV-transfer with ongoing prefill compute, without having to model that overlap explicitly.
- `KVJSQScheduler`/`KVRoundRobinScheduler` (config-driven variants; see `configs/orchestrator_repo/schedulers/`) apply similar prompt/token pool logic with different selection policies.
- `MixedPoolScheduler`: the most elaborate — a *dynamic* hybrid where Instances can be relabeled between "prompt", "token", and "mixed" pools at runtime based on load, falling back to running a request entirely on one instance ("baseline mixed batching") if neither a clean prompt nor token instance is available. It tracks `is_memory_loaded()` and `is_queue_long()` heuristics to decide when to "borrow" an instance from one pool into the `mixed_instances` pool, and `notify_free_instance()` (called by the Instance itself, via `Scheduler.notify_free_instance()`) to return a mixed instance to its home pool once idle.

9.4 Why the scheduler mutates the DAG instead of the Instance choosing

This is a deliberate separation of concerns: *scheduling policy* (which machine gets which task, whether KV-cache is even shipped at all) lives entirely in `scheduler.py`, decoupled from *execution mechanics* (how a task actually runs/batches once it's been assigned —

that's (`instance.py`)'s job). Swapping (`applications.0.scheduler=token_jsq`) for (`applications.0.scheduler=kv_token_jsq`) is the entire difference between running "Baseline" and "Splitwise" experiments, with no changes to (`Instance`) or (`Executor`) code.

10. The Performance Model: How Durations Are Estimated

Since no real neural net runs, (`performance_model.py`) is what converts "a batch of these tasks, on this hardware, with this model and parallelism" into a **wall-clock duration in seconds**. The abstract interface:

```
python

class PerformanceModel(ABC):
    def get_duration(self, task, batch, instance): ...      # one Task's to
    def get_iteration_duration(self, batch, instance): ...  # one continuou
```

10.1 (`ConstantPerformanceModel`)

Returns a fixed (`prompt_time`) or (`token_time`) regardless of anything else. Purely for smoke-testing/debugging ((`performance_model=constant`), the repo default).

10.2 (`DatabasePerformanceModel`) — the realistic one

Loads (`data/perf_model.csv`), a table of **real profiling runs** on DGX-A100 / DGX-H100 hardware, columns including (`model`), (`hardware`), (`tensor_parallel`), (`prompt_size`), (`batch_size`), (`token_size`), (`prompt_time`), (`token_time`) (converted from ms to seconds on load). Example rows:

```
model,hardware,prompt_size,batch_size,token_size,peak_power,average_power,prompt_time,token_time
llama2-70b,a100-80gb,512,1,128,1.0,0.736,196.25,54.88,7168.98,2
```

At (`init_predictor()`) time, for every (`(model, hardware, tensor_parallel)`) combination present in the CSV, it fits a **1-D linear interpolator** (`(scipy.interpolate.interp1d(..., fill_value="extrapolate"))`) mapping (`batch_tokens`) (= (`prompt_size × batch_size`) for prefill, or total decode tokens in the batch) → measured (`prompt_time`) / (`token_time`). This is a deliberately simple model: instead of trying to build a full analytical roofline/FLOP model of the GPU, it just **interpolates between empirically measured data points**, extrapolating linearly outside the measured range if needed. If an exact (`(model, hardware, tensor_parallel, batch_tokens)`) combination isn't in the DB, (`predict_new_row()`) computes it via the interpolator and **caches it back into** (`self.db`) so repeated lookups are O(1) after the first.

(`get_iteration_duration(batch, instance)`) (the function real (`ORCAInstance`) / (`SplitwiseInstance`) behavior calls) branches on batch composition:

- All-prompt batch → use the prompt-time predictor on total prompt tokens in the batch.
- All-token batch → use the token-time predictor on total decode tokens.
- **Mixed batch** (some prompts, some tokens together — happens in non-split "baseline" mode) → use the prompt predictor's estimate, then multiply by a fixed **1.1× penalty** as a simple approximation of the extra overhead of mixing phases in one batch (rather than modeling the exact hybrid-batch cost).

10.3 Why this matters for the Splitwise story

This 1.1× "mixing penalty" is a small but important detail: it's the simulator's proxy for the real-world phenomenon Splitwise is designed to avoid — mixed prompt/token batches interfering with each other. A more sophisticated performance model could measure this interference directly (and the code explicitly separates the concern into an injectable `PerformanceModel`, so you *could* swap in something more precise for your own hardware/model without touching `Instance` code at all).

11. Executors: Walking the Request DAG

`executor.py`'s `Executor` is the glue that walks a `Request`'s DAG node-by-node once the `Scheduler` has decided instance assignments. One `Executor` is spawned per request (`Scheduler.spawn_executor()`), and its job is purely mechanical:

python



```
def run(self):
    self.submit(self.request.root_node)    # kick off the first node (usually

def submit_task(self, task, instance=None):
    schedule_event(self.overheads.submit_task, lambda: instance.task_arrival(task))
    self.submit_chain(task.chain)          # also submit anything explicitly c

def finish_task(self, task, instance):
    successor_nodes = list(self.successors(task))
    if len(successor_nodes) == 0:
        self.finish_request()              # leaf node reached — request done
        return
    for node in successor_nodes:
        if node.state == NodeState.NONE and self.check_predecessors(node):
            self.submit(node)              # submit the next node(s) in the D
```

Notably: `check_predecessors()` guards against submitting a node before *all* its DAG predecessors finish — but the code comments candidly note the executor currently assumes the DAG is effectively a **chain/tree**, not a general DAG with fan-in, since it doesn't wait on multiple predecessors explicitly beyond this check. That's fine for today's request

shapes (`(prompt → token)` or `(prompt → kv_flow → token)`), but is called out as a known simplification if you wanted, e.g., multi-prompt fan-in requests.

Two `ExecutorType`s exist as scaffolding for future work: `CentralExecutor` (conceptually runs on/near the Scheduler) and `LocalExecutor` (conceptually runs on the Server alongside the Instance) — today both are empty subclasses (`(pass)`) with identical behavior, since the code doesn't yet model different overheads for where an Executor "logically" lives; the type exists so that different overhead profiles could be added later without changing the interface.

12. Allocators, Arbiters, Routers: The Outer Loop

These three exist to make the system **extensible toward autoscaling and multi-tenant clusters**, even though today's shipped implementations are largely no-ops:

- `Router` (`(router.py)`): cluster-level entry point for all requests; `NoOpRouter.route()` just forwards a request to the right Application's Scheduler after a fixed `routing_delay`. A smarter router could load-balance requests *across* multiple Applications running the same model.
- `Arbiter` (`(arbiter.py)`): cluster-level resource cop; `allocate()`/`deallocate()` methods exist for shifting `Processor`s between `Application`s, but `NoOpArbiter` does nothing. This is the extension point for **cross-application autoscaling** (e.g., "application A is overloaded, steal 2 GPUs from application B").
- `Allocator` (`(allocator.py)`): application-level instance lifecycle manager — `start_spin_up_instance()` / `finish_spin_up_instance()` (and the down-scaling counterparts, currently stubs) construct actual `Instance` objects on given `Processor`s after a configurable `overheads.spin_up` delay. `NoOpAllocator` relies entirely on **start states** (§13) to pre-populate all instances before the simulation begins, rather than dynamically spinning them up/down in response to load — dynamic instance spin-up/down is explicitly called out in the README as **not currently supported**, a known limitation/future-work item.

The existence of these no-op layers, fully wired into the event/config system, is a strong hint about the codebase's design philosophy: build the seams for autoscaling and multi-application routing now, even if the paper's experiments didn't need them, so future users can implement smarter policies by subclassing `Router`/`Arbiter`/`Allocator` without touching the simulator core.

13. Start States: How a Simulation Begins

Because dynamic instance spin-up isn't supported, every simulation needs a **starting layout** of Instances pre-placed on Servers — this is `start_state.py`'s job, invoked once via `load_start_state(cfg.start_state, ...)` right after the `TraceSimulator` is constructed but before `.run()`.

Three flavors, chosen by `start_state_cfg.state_type`:

1. `unallocated`: literally nothing pre-placed (for scenarios where you build your own custom setup).
2. `orca` / `baseline` → `uniform()`: put exactly **one Instance per Server**, all running the same model/parallelism config, untagged. This is the "everything does both phases" baseline.
3. `splitwise*` → `splitwise()`: separate **prompt** and **token** Instance configs (each can have its own parallelism!), tagged `"prompt"` or `"token"`, with two placement strategies:
 - `split_type: homogeneous` — take the first N servers (any SKU) for prompt instances, next M for token instances, by count.
 - `split_type: heterogeneous` — explicitly map SKU names to prompt vs. token role (e.g., "all DGX-H100s are prompt machines, all DGX-A100s are token machines" — this is exactly the `Splitwise-HA` configuration from the paper).

All instances are spun up with `pre_start=True`, meaning `finish_spin_up_instance()` runs immediately/synchronously rather than waiting through a `schedule_event` delay — the cluster is "born" fully staffed at simulation time 0.

14. End-to-End Walkthrough: The Life of One Request

Tying §§6-13 together, here's exactly what happens for one row of a trace CSV, assuming a Splitwise-style scheduler:

1. **Load:** `Trace.from_csv()` reads the row, `Request.from_dict()` builds a `GenerativeLLMRequest`, whose `__post_init__` creates a `PromptTask` and `TokenTask` and wires `prompt_task → token_task` in its `networkx` DAG.
2. **Arrival event:** `TraceSimulator.load_trace()` had already scheduled `router.request_arrival(request)` to fire at `request.arrival_timestamp`.
3. **Router:** `Router.request_arrival()` calls `request.arrive_at_router()` (state → `QUEUED_AT_ROUTER`, timestamp stamped), then `route_request()` → `NoOpRouter.route()` schedules `scheduler.request_arrival(request)` after a fixed `routing_delay`.
4. **Scheduler:** `Scheduler.request_arrival()` calls `request.arrive_at_scheduler()`, adds it to `pending_queue`, and — since schedulers here process one request at a time as they arrive (`if len(self.pending_queue) == 1: self.run_request(request)`) — immediately (or once free) calls `run_request()`.
5. `run_request()`: calls `request.run_on_executor()` (state → `RUNNING_ON_EXECUTOR`), then the scheduler's own `schedule(request)` method — e.g. `KVTokenJSQScheduler.schedule()` — picks a `prompt_instance` and `token_instance`, and calls `add_kv_cache_transfer()`, which **rewrites the DAG** to

`prompt_task → kv_flow → token_task` and assigns `prompt_task.instance / token_task.instance`.

6. **Executor spawned:** `spawn_executor()` creates a fresh `Executor` for this request and calls `.run()`, which submits the `root_node` (`prompt_task`) — scheduling a `task_arrival` event on the prompt Instance after a small `submit_task` overhead delay.
7. **Prompt Instance runs it:** the `SplitwiseInstance.task_arrival()` adds the task to its pending/pool queues, and (if idle) calls `start_iteration()`, which selects a batch (possibly including other requests' prefill tasks), asks the `PerformanceModel` for `get_iteration_duration()`, and schedules a `complete_iteration()` event.
8. **Prefill completes:** `complete_iteration()` marks `generated_tokens=1` on the prompt task → `is_complete()` is true → `task_completion()` runs → `task.complete()` stamps `TTFT` (time-to-first-token) on the request → `executor.finish_task()` looks at DAG successors, finds the `KVCacheTransferFlow`, and — since its only predecessor (the prompt task) is now complete — submits it.
9. **Flow runs on a Link:** `submit_flow()` schedules `link.flow_arrival(flow)`; the (Dummy)Link computes `flow.duration = flow.size / bandwidth` and schedules `flow_completion()`.
10. **KV-cache "arrives":** `flow.complete()` frees memory on the source Instance; `executor.finish_flow()` finds the `token_task` successor (predecessor now complete) and submits it to the token Instance.
11. **Token Instance decodes:** same `task_arrival` → `start_iteration()` → repeated `complete_iteration()` cycle as step 7-8, except now running many contiguous decode iterations at once (since a pure-token batch can run several iterations before needing re-selection), until `generated_tokens == token_size`.
12. **Request completes:** the token task's `complete()` has no DAG successors, so `executor.finish_request()` fires, calling `scheduler.request_completion(request)` → `request.complete_at_scheduler()` (stamps final response time) → `router.request_completion(request)` → `request.complete_at_router()`.
13. **Metrics saved:** at the very end of the whole simulation, `TraceSimulator.save_results()` aggregates every completed request's metrics into `summary.csv` and `detailed/{app_id}.csv`.

Every timestamp mentioned above (`router_arrival_timestamp`, `scheduler_arrival_timestamp`, `prompt_end_timestamp`, `TTFT`, `router_response_time`, ...) is a real field on `RequestMetrics`/`GenerativeLLMRequestMetrics` (`metrics.py`) that gets written to the CSV outputs — this is the raw material for every plot in the paper.

15. Running a Simulation, Concretely

15.1 Setup

```
bash

conda create -n splitwise-sim python=3.11
conda activate splitwise-sim
pip install -r requirements.txt
```

15.2 Simplest possible run

```
bash

python run.py
```

This uses every default in `config.yaml`: `dgx-a100` cluster, `noop` router/arbiter, `solo` application (round-robin scheduler, bloom-176b model), `test_trace`, `orca` start-state, `constant` performance model.

15.3 The example scripts

```
bash

./scripts/run_baseline_h_example.sh      # single DGX-H100, mixed-batch baseline
./scripts/run_splitwise_ha_example.sh    # DGX-H100 (prompt) + DGX-A100 (token),
```

Must be run from the repo's top-level directory (they reference relative config paths).

15.4 `run.py`'s orchestration

```
python
```

```
def run_simulation(cfg):
    hardware_repo = init_hardware_repo(cfg)
    model_repo = init_model_repo(cfg)
    orchestrator_repo = init_orchestrator_repo(cfg)
    performance_model = init_performance_model(cfg)
    power_model = init_power_model(cfg)
    cluster = init_cluster(cfg)
    router = init_router(cfg, cluster)
    arbiter = init_arbiter(cfg, cluster)
    applications = init_applications(cfg, cluster, router, arbiter)
    trace = init_trace(cfg)
    for application in applications.values():
        router.add_application(application)
        arbiter.add_application(application)
    sim = TraceSimulator(trace, cluster, applications, router, arbiter, end_time)
    init_start_state(cfg, cluster=cluster, applications=applications, router=router)
    sim.run()
```

This function is a clean, linear "wire everything up, then press play" — every `init_*` in `initialize.py` is a small factory function converting one config sub-tree into the corresponding live object (registries first, since later objects need to look things up in them; the cluster/applications/trace next; the start state last, since it needs live `Instances` `Allocators` to already exist).

15.5 Parallel/multirun

```
bash

ray start --head # on a head node
python --multirun run.py seed=0,1,2,3,4,5,6,7,8,9
```

Each parameter combination in a `--multirun` sweep runs as an **independent single-core simulation process**; Ray (or the joblib launcher, single-machine only) just handles scheduling those processes across cores/machines. Multi-machine output must be synced back manually (`sync_scripts/`).

16. Outputs and Metrics

Every run writes into a Hydra-managed working directory (path template configured by `output_dir` in `config.yaml`), keyed by seed/start-state/trace/cluster-mix/model/scheduler — so different configs never clobber each other's results):

File	Contents
<code>summary.csv</code>	One row per application; mean/median/p50/p90/p99/etc. (via <code>utils.get_statistics()</code>) of response time, queue time, TTFT, time-between-tokens, prompt/token sizes
<code>detailed/{application_id}.csv</code>	One row per completed request , with the same metrics unaggregated — this is the raw data behind every per-request analysis
<code>detailed/{application_id}_alloc.csv</code>	Per-instance utilization stats from the <code>Allocator</code>
<code>request_nodes.csv</code>	One row per DAG node (every Task/Flow of every completed request) with <code>start_timestamp</code> / <code>completion_timestamp</code> /which Instance or Link it ran on — this is what Gantt-chart plotting uses
<code>instances/{app_id}/{instance_id}.log</code> (only if <code>debug=True</code>)	Per-iteration Instance state: batch size, memory, pending/blocked queue lengths, batch tokens, etc. — the finest-grained trace available
<code>oom.csv</code> (only if it happens)	Logged whenever a <code>Processor</code> 's <code>memory_used</code> setter detects memory oversubscription

`notebooks/utils.py` provides helper functions to load and reshape these CSVs; `notebooks/example.ipynb` shows a basic load-and-plot workflow (including Gantt charts from `request_nodes.csv` + the debug instance logs); `notebooks/plots.ipynb` reproduces several of the actual paper figures.

17. Advanced: Preemption and Batch Selection Logic

This is the trickiest part of the codebase, so it's worth walking through `SplitwiseInstance.select_batch()` (`instance.py`) explicitly, since it encodes real scheduling *policy*, not just mechanism. Its job: given the current `pending_queue` (tasks not yet running) and `old_batch` (tasks that were running last iteration), decide the `new_batch` for this iteration, and which `old_batch` tasks must be **preempted** (evicted) to make room. It builds `new_batch` in **strict priority order**, stopping each stage once `max_batch_size` or `max_batch_tokens` would be exceeded:

1. **Starvation-guard tasks first:** any pending task that has already been preempted `>= max_preemptions` times is force-included (guarantees forward progress for unlucky requests).

2. **New prompt tasks** (`pending_prompt_queue`): prompts get priority over continuing decodes, since the paper's design deliberately favors low time-to-first-token — this is also literally what causes decode-task preemption in the first place.
3. **Already-blocked tasks** (`blocked_queue`): tasks that were preempted in a *previous* round and are waiting to resume, added before *this* round's yet-untouched old batch — so a request doesn't get preempted repeatedly while other requests cut ahead of it.
4. **Remaining old-batch decode tasks**: tasks that were already running and aren't displaced.
5. **Any other pending decode task**, memory permitting.

python

```
preempted_tasks = [task for task in old_batch if task not in new_batch]
new_tasks       = [task for task in new_batch if task not in old_batch]
```

Back in `task_arrival()`, this preemption is only *triggered* under specific conditions — a new prompt task arrives, the instance is fully busy, but there's room within `max_batch_tokens` if some in-flight decode tasks are evicted:

python

```
if isinstance(task, PromptTask):
    batch_prompt_tokens = self.batch_tokens - len(self.token_tasks_in_batch)
    if len(self.prompt_tasks_in_batch) < len(self.batch) and \
        batch_prompt_tokens + task.prompt_size <= self.max_batch_tokens:
        self.preempt_iteration()
```

`preempt_iteration()` doesn't do the preemption itself — it just triggers an early re-evaluation at the next safe iteration boundary (identical mechanism to `pause_iteration()` in the ORCA base class, via `reschedule_event()`), and the *next* call to `start_iteration()` → `select_batch()` is what actually works out and applies the eviction. This separation (schedule an early boundary vs. decide what happens at that boundary) is what keeps the "when" and "what" of preemption decoupled and independently testable.

Preempted `TokenTask`s cannot lose partial iteration progress below iteration granularity — the whole design assumes decode always operates in whole-iteration increments, so `task.preempt()` simply records a timestamp and moves the task back to `pending_queue`/`blocked_queue`; there's no partial-token state to roll back.

Note: `PromptTask`s are explicitly disallowed from being preempted (`raise ValueError("Prompt tasks cannot be preempted")` in `preempt_task()`) — once prefill starts, it always runs to completion. Only decode tasks can be evicted.

18. Advanced: Memory and KV-Cache Accounting

Memory is tracked at **two different granularities** simultaneously, and it's worth being precise about the difference, since the codebase itself calls this out with a `(TODO: finer-grained memory management)` comment:

1. **Actual memory** (`(Instance.memory) / (alloc_memory) / (free_memory)`): incremented when a `(Task) / (Flow)` actually starts running (`(Task.run())` calls `(instance.alloc_memory(self.request, self.memory))`), decremented when it completes (if `(cleanup_memory=True)`). This is what's checked against `(max_memory)` before admitting a task into a running batch (`(select_batch)`'s `(if task.memory + memory <= self.max_memory)`) — it's the constraint that determines OOM risk in the here-and-now of the simulation.
2. **Scheduler-level memory** (`(sched_memory) / pending tokens` (`(sched_pending_tokens)`): bumped *at scheduling time* (before the task even physically starts), used only by `(Scheduler)` implementations like `(MixedPoolScheduler)` to make load-balancing decisions (`(is_memory_loaded())`) — a forward-looking estimate of how loaded an instance *will be*, used to route new requests sensibly, decoupled from the instant-by-instant `(Instance.memory)` bookkeeping used for actual admission control.

The KV-cache size formula (`(Request.estimate_kv_cache_size)`, §6.5) is central to both: `(PromptTask.memory)` uses it for `(prompt_size + 1)` tokens (the +1 accounts for the first generated token's KV entry, produced as a side-effect of prefill), while `(TokenTask.memory)` uses it for `(token_size)` tokens. `(max_memory(instance))` variants on both Task types let a *Scheduler* (which doesn't yet know which Instance a task will land on) ask "how much memory would this task need *if* placed on this specific candidate instance" — useful because different instances might run the model with different parallelism (and hence different per-GPU memory footprint), per `(model.py)`'s `(size_per_processor)` concept.

`(cleanup_memory)` (a flag on `(Task)`, defaulting to `(True)` on `(TokenTask)` and `(False)` on `(PromptTask)`) governs whether completing that task frees the request's KV-cache memory — it's `(False)` on `(PromptTask)` because the KV-cache must obviously **survive** into the decode phase; it only gets freed once the whole request (i.e. its final `(TokenTask)`) completes.

19. Extending SplitwiseSim

Because of the registry/config pattern (§5, §7.5), most extensions require **zero changes to the simulation engine itself** — you add a config file and, if needed, a small Python class. Concretely:

- **Add a new scheduling policy:** subclass `(Scheduler)` (or `(KVScheduler)` if it's KV-transfer-based) in `(scheduler.py)`, implement `(schedule())`, then add a YAML file under `(configs/orchestrator_repo/schedulers/)` with a `(target_)` pointing at your

class. Now `applications.0.scheduler=your_new_name` works from the command line.

- **Add new hardware:** add a processor YAML under `configs/hardware_repo/processors/`, a server SKU YAML under `configs/hardware_repo/skus/` referencing it, and profile+append rows to `data/perf_model.csv` for that hardware/model combination (or implement a smarter `PerformanceModel` if interpolation isn't precise enough for your hardware).
 - **Add a new model:** add architecture/size YAMLs under `configs/model_repo/`.
 - **Add a new instance execution strategy** (e.g., chunked prefill, speculative decoding): subclass `Instance`/`ORCAInstance`/`SplitwiseInstance` in `instance.py`, add the corresponding branch in `Instance.from_config`, and reference the new `instance_type` from a `start_state` YAML.
 - **Add smarter routing/autoscaling:** subclass `Router`/`Arbiter`/`Allocator` — the interfaces `(route(), allocate(), deallocate(), start_spin_up_instance(), start_spin_down_instance())` already exist; today's shipped versions are just no-ops/static, by design, waiting for exactly this kind of extension.
 - **Generate custom traces:** `generate_trace.py` can download the referenced AzurePublicDataset LLM inference traces and resample prompt/token-size distributions at different request rates, or you can hand-write a CSV following the 7-column schema in §6 as long as it matches the `traces/test_trace.csv` format.
-

20. File-by-File Reference Map

File	Role
<code>simulator.py</code>	Discrete-event engine: <code>Event</code> , <code>Simulator</code> , <code>TraceSimulator</code> , global <code>clock()</code> / <code>schedule_event()</code>
<code>request.py</code>	<code>Request</code> / <code>GenerativeLLMRequest</code> : DAG-of-nodes request model, state machine, KV-cache size estimate
<code>node.py</code>	<code>Node</code> base class shared by <code>Task</code> and <code>Flow</code> ; state machine + metrics timestamps
<code>task.py</code>	<code>Task</code> / <code>PromptTask</code> / <code>TokenTask</code> : compute nodes, iteration-level token bookkeeping
<code>flow.py</code>	<code>Flow</code> / <code>KVCacheTransferFlow</code> : communication nodes that run on <code>Links</code>
<code>executor.py</code>	<code>Executor</code> : walks a request's DAG, submitting nodes to Instances/Links as predecessors complete
<code>instance.py</code>	<code>Instance</code> / <code>ORCAInstance</code> / <code>SplitwiseInstance</code> : continuous batching, preemption, memory admission control
<code>scheduler.py</code>	<code>Scheduler</code> and all concrete policies (<code>RoundRobin</code> , <code>JSQ</code> , <code>KVTokenJSQ</code> , <code>MixedPool</code> , ...)
<code>allocator.py</code>	<code>Allocator</code> / <code>NoOpAllocator</code> : spins Instances up/down onto Processors
<code>router.py</code>	<code>Router</code> / <code>NoOpRouter</code> : cluster-level dispatch of Requests to Applications
<code>arbiter.py</code>	<code>Arbiter</code> / <code>NoOpArbiter</code> : cluster-level resource rebalancing across Applications
<code>application.py</code>	<code>Application</code> : an endpoint bundling a model + scheduler + allocator + instances
<code>cluster.py</code>	<code>Cluster</code> : collection of Servers + interconnects + power budget
<code>server.py</code>	<code>Server</code> : collection of Processors + local interconnects, built from SKU configs

File	Role
<code>processor.py</code>	<code>Processor</code> / <code>CPU</code> / <code>GPU</code> : lowest-level compute unit, memory/power tracking
<code>interconnect.py</code>	<code>Link</code> and subtypes (<code>NVLink</code> , <code>IBLink</code> , <code>DummyLink</code> , ...): where Flows run
<code>model.py</code>	<code>Model</code> / <code>GenerativeLLM</code> , <code>ModelArchitecture</code> , <code>ModelParallelism</code> , <code>ModelSize</code> dataclasses
<code>performance_model.py</code>	<code>PerformanceModel</code> : <code>ConstantPerformanceModel</code> , <code>DatabasePerformanceModel</code> (interpolated from profiling CSV)
<code>power_model.py</code>	<code>PowerModel</code> : <code>ConstantPowerModel</code> , <code>DatabasePowerModel</code> (largely unused/TODO)
<code>metrics.py</code>	All <code>@dataclass</code> metric/SLO containers stamped throughout the simulation
<code>hardware_repo.py</code>	Registry of processor/interconnect/SKU configs, for dynamic instantiation
<code>model_repo.py</code>	Registry of model architecture/size configs
<code>orchestrator_repo.py</code>	Registry of scheduler/allocator configs
<code>start_state.py</code>	<code>load_start_state</code> : pre-populates the cluster with Instances before t=0 (<code>uniform</code> , <code>splitwise</code>)
<code>trace.py</code>	<code>Trace</code> : loads a request-arrival CSV into a list of <code>Request</code> objects
<code>initialize.py</code>	<code>init_*</code> factory functions wiring config → live objects
<code>run.py</code>	Hydra entrypoint (<code>@hydra.main</code>); builds everything via <code>initialize.py</code> and calls <code>sim.run()</code>
<code>generate_trace.py</code>	Downloads/derives request traces from Azure production trace distributions
<code>utils.py</code>	Logging helpers, CSV saving, statistics helpers
<code>configs/</code>	All Hydra YAML configuration (see §5)
<code>data/perf_model.csv</code>	Profiled DGX-A100/DGX-H100 measurements backing <code>DatabasePerformanceModel</code>

File	Role
<code>traces/test_trace.csv</code>	A small example request trace
<code>scripts/</code>	Shell scripts for individual runs and full experiment sweeps
<code>notebooks/</code>	<code>example.ipynb</code> (basic usage + Gantt charts), <code>plots.ipynb</code> (paper figures), <code>utils.py</code> (result-loading helpers)
<code>sync_scripts/</code>	Helpers to collect results from multi-machine Ray runs

21. Glossary

- **Prefill / prompt phase:** the parallel, compute-bound pass over all input tokens that produces the first output token.
- **Decode / token phase:** the serial, memory-bound generation of each subsequent output token.
- **KV-cache:** the per-layer, per-token Key/Value tensors cached so each decode step needn't recompute attention over the whole sequence; its size scales with sequence length and must live in GPU memory for the duration of a request.
- **Continuous / iteration-level / in-flight batching:** a serving strategy (popularized by ORCA, used by vLLM etc.) where the "batch" running on a GPU is re-evaluated at every model-forward-pass boundary, so new requests can join and finished requests can leave without waiting for the whole batch to drain.
- **Preemption:** temporarily evicting a running (decode) task from the batch to make room for a higher-priority task (typically an incoming prefill), to be resumed later.
- **TTFT:** Time To First Token — latency from request arrival to the first generated token; dominated by queueing + prefill.
- **TBT:** Time Between Tokens — average per-token latency during decode.
- **Splitwise-HA / Splitwise-HH / Baseline-H / Baseline-A:** naming convention used across `scripts/`/paper for cluster configurations — letters denote hardware SKU for prompt vs. token pools (H = DGX-H100, A = DGX-A100); e.g. Splitwise-HA = H100s do prompts, A100s do tokens.
- **Iso-cost / iso-power sweeps:** experiment methodology comparing cluster configurations that are equalized on total dollar cost or total power draw, to isolate the effect of the Splitwise design itself rather than "more hardware wins."
- **Hydra:** the Python configuration-management library (`hydra.cc`) used for hierarchical YAML config, command-line overrides, and multi-run sweeps.
- **DES:** Discrete-Event Simulation — the simulation paradigm underlying `simulator.py` (see §3).

This document was written by inspecting the actual `splitwise-sim` source tree (cloned from GitHub) alongside its README and the accompanying ISCA 2024 paper's public description, so that code snippets and behavior described here match the real implementation rather than a general/idealized description of Splitwise.